

1.2 Numpy数组的运算

1.2.1 向量的运算

1. 对元素的操作

如果要进行逐元素的运算就可能需要用到循环。和列表类似，我们可以利用for循环来遍历一个numpy.array。例如：

```
import numpy as np

# 创建一个Numpy数组
arr = np.array([1, 2, 3, 4, 5])

# 使用for循环遍历数组
for i in arr:
    print(i)
```

但是，如果在numpy当中你仍然大规模使用循环的话，它的效率提升不了一点。我们发明numpy的初衷就是希望能够以向量化并行计算的方式去提升运算效率。这里我们介绍几种利用向量批量操作的方法。

第一个我们在上一节当中提到。numpy.arange([start,]stop, [step,]dtype=None) 函数类似于Python内置的 range() 函数，但返回的是一个数组，而不是一个迭代器或列表。它生成一个给定起始值 start（默认为0）、结束值 stop（不包括此值）和步长 step（默认为1）的数组。

```
# 使用numpy.arange
arr = np.arange(0, 10, 2) # 类似于range(0, 10, 2)
print(arr) # 输出: [0 2 4 6 8]
```

第二个方法

是 numpy.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0) 函数，用于生成一个等差数列，即在指定的间隔内返回均匀间隔的数字。与 numpy.arange 不同，numpy.linspace 是基于间隔数量（num）来生成数组的，而不是步长。

```
# 使用numpy.linspace
arr = np.linspace(0, 10, 5) # 在0到10之间生成5个等间距的点
print(arr) # 输出类似: [ 0.  2.5  5.  7.5 10.]
```

numpy.arange、numpy.linspace与range之间的用法存在一些互通之处，但也有不同点。它们都可以用来生成一系列的数字。不同点在于：

- **类型**： range 返回的是一个迭代器，而 numpy.arange 和 numpy.linspace 返回的是Numpy数组。
- **用途**： range 主要用于Python的for循环中迭代，而 numpy.arange 和 numpy.linspace 主要用于Numpy的数值计算中，特别是在需要数组操作时。
- **参数**： numpy.arange 和 range 基于起始值、结束值和步长生成序列，而 numpy.linspace 基于起始值、结束值和元素数量生成等差数列。
- **性能**： 在处理大型数据集时，Numpy数组（ numpy.arange 和 numpy.linspace 生成的）比Python列表（ range 转换为列表）或迭代（直接使用 range ）在性能上有显著优势，因为Numpy数组在内存中连续存储，并且支持向量化操作。

2. 向量的线性运算

好了，现在让我们切入今天的正题，就是如何使用np.array模拟一个向量。那么大家在中学阶段学的向量主要是二维欧式空间或三维欧式空间下的向量对吧，如果我告诉你一个向量的维度可以有更多维，大家还可以理解吗？哈哈，其实在numpy当中，一个一维数组就可以表示一个n维向量了。向量的概念其实是可以扩展到更高维空间当中的。

想象一下，我们手里拿着一堆乐高积木（其实就是Numpy数组啦，不过乐高更有趣点），现在我们要用这些积木来玩点“数学魔法”——向量的加减法和缩放运算。准备好了吗？魔法即将开始！

首先，我们有两个小伙伴，小明和小华，他们各自手里拿着一串乐高积木，代表两个向量。小明的向量是 [1, 2, 3]，而小华的是 [4, 5, 6]。他们想一起玩，所以决定把各自的积木加在一起。

在Numpy的世界里，这超级简单：

```
import numpy as np

# 小明的向量
ming_vector = np.array([1, 2, 3])
# 小华的向量
hua_vector = np.array([4, 5, 6])

# 向量加法
combined_vector = ming_vector + hua_vector

print("小明和小华的乐高积木加在一起变成了: ", combined_vector)
# 输出: 小明和小华的乐高积木加在一起变成了:  [5 7 9]
```

看，就像把两堆乐高积木放在一起那么简单！

不过，有时候小朋友们也会闹别扭，比如小明觉得小华手上的乐高太多了，想要抢一点过来。这时候，我们就需要做减法了。

```
# 向量减法
difference_vector = hua_vector - ming_vector

print("小华从小明那里抢了一些乐高后，剩下的乐高是: ", difference_vector)
# 输出: 小华从小明那里抢了一些乐高后，剩下的乐高是:  [3 3 3]
```

接下来，假设我们有一个神奇的放大镜，可以让我们手上的乐高积木变大或变小。这个放大镜就是“缩放运算”。

比如，我们用放大镜把小明手上的乐高积木都放大2倍：

```
# 缩放运算，放大2倍
scaled_ming_vector = ming_vector * 2

print("小明用放大镜把乐高积木放大2倍后，得到了: ", scaled_ming_vector)
# 输出: 小明用放大镜把乐高积木放大2倍后，得到了:  [2 4 6]
```

通过这些简单的例子，我们可以看到，使用Numpy进行向量的加减法和缩放运算，就像玩乐高一样轻松愉快。在数学和计算机科学中，向量是基本且重要的概念，而Numpy库提供了强大的工具来处理这些操作。

对于任意两个维度相同的向量 $\vec{a} = (a_1, a_2, \dots, a_n)$ 和 $\vec{b} = (b_1, b_2, \dots, b_n)$ ，它们的加法定义为：

$$\vec{a} + \vec{b} = (a_1 + b_1, a_2 + b_2, \dots, a_n + b_n)$$

它们的减法定义为：

$$\vec{a} - \vec{b} = (a_1 - b_1, a_2 - b_2, \dots, a_n - b_n)$$

在Numpy中，这些操作可以直接通过 `+` 和 `-` 运算符完成。

向量的缩放运算涉及一个标量（即一个单一的数）和一个向量。给定一个标量 k 和一个向量 $\vec{a} = (a_1, a_2, \dots, a_n)$ ，向量的缩放定义为：

$$k\vec{a} = (ka_1, ka_2, \dots, ka_n)$$

在Numpy中，这可以通过将标量与向量相乘来实现。

对于更高维度的向量（例如四维或更高），Numpy的处理方式与上述相同。只需确保在进行运算时，涉及的向量维度相同即可。

```
# 定义两个四维向量
vec_c = np.array([1, 2, 3, 4])
vec_d = np.array([5, 6, 7, 8])

# 向量加法
vec_sum_4d = vec_c + vec_d
print("四维向量加法结果：", vec_sum_4d) # 输出：四维向量加法结果： [6 8 10 12]

# 向量缩放（以标量3为例）
scaled_vec_4d = 3 * vec_c
print("四维向量缩放结果（乘以3）：", scaled_vec_4d) # 输出：四维向量缩放结果（乘以3）： [3 6 9 12]
```

3. 向量的非线性运算

加减法和数乘都属于是比较基础的线性运算，但向量还有一些非线性运算方法。**向量的数量积（又称点积、内积或标量积）**是一种特殊的向量运算，其结果是一个标量（即一个单一的数），而不是向量。向量的数量积定义为两个向量的大小、它们之间夹角的余弦值的乘积。具体地，对于两个向量 $\vec{a}$ 和 $\vec{b}$ ，它们的数量积表示为 $\vec{a} \cdot \vec{b}$ 或 $a \cdot b$ ，计算公式为：

$$\vec{a} \cdot \vec{b} = |\vec{a}| \times |\vec{b}| \times \cos \theta$$

其中， $|\vec{a}|$ 和 $|\vec{b}|$ 分别是向量 $\vec{a}$ 和 $\vec{b}$ 的模（即大小或长度）， θ 是向量 $\vec{a}$ 和 $\vec{b}$ 之间的夹角。

数量积具有一些重要的性质，例如：

- 当两个向量同向时，数量积为正；反向时，数量积为负；垂直时，数量积为零。
- 数量积满足交换律，即 $\vec{a} \cdot \vec{b} = \vec{b} \cdot \vec{a}$ 。
- 数量积的绝对值不超过两个向量模的乘积，即 $|\vec{a} \cdot \vec{b}| \leq |\vec{a}| \times |\vec{b}|$ 。

在Numpy中实现向量的数量积，可以使用 `numpy.dot()` 函数或 `@` 操作符。这两个操作在处理一维数组（可以视为向量）时，都会计算它们的点积（即数量积）。

使用 `numpy.dot()` 函数：

```
import numpy as np

# 定义两个向量
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 计算数量积
dot_product = np.dot(a, b)

print("数量积为: ", dot_product) # 输出：数量积为： 32
```

使用 `@` 操作符（Python 3.5+）：

```
import numpy as np

# 定义两个向量
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 计算数量积
dot_product = a @ b

print("数量积为: ", dot_product) # 输出：数量积为： 32
```

在这两个示例中，`a` 和 `b` 是两个三维向量，我们通过Numpy的 `dot()` 函数或 `@` 操作符计算了它们的数量积，并打印了结果。注意，虽然这两个操作符在处理一维数组时等同于计算数量积，但它们在处理多维数组时会有不同的行为（例如，`@` 操作符在Python中用于矩阵乘法，而 `numpy.dot()` 在处理二维数组时也会进行矩阵乘法，但在处理更高维数组时行为较为复杂）。然而，在本例中，我们仅关注一维数组（向量）的情况。

这里可能有同学要问了，为什么这里不能直接用乘号呢？我们来看看这里如果改成乘号会变成什么东西：

```
array([4,10,18])
```

你会发现，结果变成了一个向量。这种计算叫逐元素乘法。那么如何把它变成正确的数量积呢？只需要对这个数组进行一趟求和就好了。

另一种非线性运算就是叉乘。向量的叉乘（Cross Product），也称为向量的叉积或向量积，是向量代数中的一个重要概念。它涉及到两个三维向量的运算，其结果是一个新的向量，这个新向量垂直于原来两个向量所构成的平面。

1. **定义**：对于两个三维向量 $\vec{a} = (a_1, a_2, a_3)$ 和 $\vec{b} = (b_1, b_2, b_3)$ ，它们的叉乘 $\vec{c} = \vec{a} \times \vec{b}$ 是一个新的向量，其方向垂直于 $\vec{a}$ 和 $\vec{b}$ 所在的平面，并且满足右手定则（即四指从 $\vec{a}$ 转向 $\vec{b}$ 时，大拇指的指向即为 $\vec{c}$ 的方向）。
2. **坐标表示**：在三维空间中，叉乘的坐标可以通过行列式的方式计算，具体为：

$$\vec{c} = \vec{a} \times \vec{b} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{vmatrix} = (a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1)$$

其中， $\mathbf{i}$ 、 $\mathbf{j}$ 、 $\mathbf{k}$ 分别是沿 x 、 y 、 z 轴的单位向量。

3. 性质：

- 叉乘不满足交换律，即 $\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$ 。
- 叉乘的模等于两向量模的乘积与它们之间夹角的正弦值的积，即 $|\vec{a} \times \vec{b}| = |\vec{a}| \cdot |\vec{b}| \cdot \sin \theta$ 。
- 叉乘的结果垂直于原来两个向量所构成的平面。

在Numpy库中，可以直接使用 `numpy.cross()` 函数来计算两个三维向量的叉乘。这个函数接受两个一维数组（代表三维向量）作为输入，并返回一个一维数组（代表叉乘结果的三维向量）。

```
import numpy as np

# 定义两个三维向量
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# 计算叉乘
c = np.cross(a, b)

# 输出结果
print("叉乘结果：", c) # 输出类似于：[ -3  6 -3]
```

在这个例子中，`a` 和 `b` 是两个三维向量，通过调用 `np.cross(a, b)` 计算它们的叉乘，并将结果存储在 `c` 中。最后，通过打印 `c` 来查看叉乘的结果。

注意：

- 叉乘是三维向量特有的运算，不适用于二维向量或更高维度的向量（尽管可以通过一些方式在二维空间中模拟叉乘的概念，但那不是真正的叉乘）。
- 在使用Numpy进行叉乘计算时，请确保输入的向量是一维数组，并且长度为3（代表三维空间中的坐标）。如果输入不符合这些要求，`numpy.cross()` 函数可能会引发错误。

在NumPy中，对数组进行排序、筛选等操作是非常常见的任务，特别是在处理科学计算和数据分析时。这里，我将详细介绍如何对NumPy数组进行排序、筛选，特别是针对多条件筛选和多维数组筛选的方法。

NumPy提供了 `numpy.sort()` 函数和数组的 `sort()` 方法对数组进行排序。`numpy.sort()` 函数返回排序后的数组副本，而数组的 `sort()` 方法则直接对数组本身进行原地排序。

```
import numpy as np

# 创建一个数组
arr = np.array([3, 1, 4, 1, 5, 9, 2])

# 使用numpy.sort()排序，返回新数组
sorted_arr = np.sort(arr)
print("Sorted array with numpy.sort:", sorted_arr)

# 使用数组的sort()方法排序，原地修改
arr.sort()
print("Array sorted in-place:", arr)
```

4. 向量的逻辑运算——筛选

筛选通常指的是基于某些条件选择数组中的元素。在NumPy中，这可以通过布尔索引、`numpy.where()` 函数、以及使用条件表达式来实现。

单条件筛选

```
# 假设我们有一个数组
arr = np.array([1, 2, 3, 4, 5, 6])

# 筛选所有大于3的元素
filtered_arr = arr[arr > 3]
print("Filtered array:", filtered_arr)
```

多条件筛选

对于多条件筛选，可以使用逻辑运算符（如 & 和 |）来组合多个条件。注意，在使用 & 和 | 时，需要确保每个条件都被包裹在括号中，并且可能需要使用 `numpy.logical_and()` 和 `numpy.logical_or()` 函数来确保逻辑操作是逐元素的。

```
# 创建一个二维数组
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# 筛选所有行，其中第一个元素大于3且第二个元素小于8
# 注意使用&和括号
filtered_rows = arr_2d[(arr_2d[:, 0] > 3) & (arr_2d[:, 1] < 8)]
print("Filtered rows:", filtered_rows)

# 或者使用numpy.logical_and()
filtered_rows_logical = arr_2d[np.logical_and(arr_2d[:, 0] > 3, arr_2d[:, 1] < 8)]
print("Filtered rows with logical_and:", filtered_rows_logical)
```

1.2.2 矩阵的运算

1. 矩阵的算术运算

矩阵就像是数学世界里的超级大表格，每个小格子里都藏着一个数字，等着我们去探索。矩阵的加减法，就像是咱们在超市买完东西后，用计算器加加减减那些商品的价格一样，只不过这次咱们是在这些大表格上动手。

想象一下，你有两个一模一样的超级大表格，只不过每个格子里的数字可能不一样。矩阵加法呢，就是让你把这两个表格对应格子里的数字加在一起，然后得到一个新的表格。就像是两个朋友一起数钱，然后把各自的零钱合在一起，看看总共有多少。

在NumPy里，做这件事超级简单。你只需要用个加号 +，把两个矩阵（在NumPy里叫做数组）放在一起，NumPy就会自动帮你把对应位置的数字加起来，然后返回一个全新的数组。比如：


```
import numpy as np

# 创建两个矩阵（数组）
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# 矩阵加法
result_add = matrix1 + matrix2
print(result_add) # 输出: [[ 6  8] [10 12]]
```

看，简单到不行吧，就像是小学生做加法题一样。

矩阵的减法呢，就像是咱们在超市买完东西后，发现有些东西买多了，想要退掉一些。不过在这里，我们不是真的退东西，而是想象你有两个表格，然后从第一个表格的每个格子里减去第二个表格对应格子的数字。这就像是你和朋友一起数钱，但这次你决定要看看自己比朋友多了还是少了多少钱。

在NumPy里，做矩阵减法也是用个简单的减法符号 `-`，把两个矩阵放在一起，NumPy就会帮你把对应位置的数字相减，然后返回结果。比如：

```
# 接着上面的例子
result_subtract = matrix1 - matrix2
print(result_subtract) # 输出: [[-4 -4] [-4 -4]]
```

看，这就像是玩游戏里的“找不同”，只不过我们是找“差多少”。

矩阵的加减法在NumPy里就是这么简单直白，就像是我们平时做算术题一样。NumPy就像是那个超级智能的计算器，帮我们把那些复杂的数字运算变得轻而易举。所以，下次当你看到矩阵的加减法时，别害怕，想象成是在玩一个高级的“数独”游戏，只不过这次你是用加减号来填满格子，而不是数字。

矩阵的加减法**定义**：

设A和B是两个同型矩阵（即行数和列数分别相等），则矩阵A与B的和（或差）是一个与A、B同型的矩阵，记为A+B（或A-B），其元素由A和B对应元素相加（或相减）得到。

运算法则与数学公式：

- 加法交换律：** $A + B = B + A$ 。
 - 这意味着矩阵加法的顺序不影响最终结果。
- 加法结合律：** $(A + B) + C = A + (B + C)$ 。
 - 类似于数的加法，矩阵的加法也满足结合律。
- 零矩阵与加法恒等元：** $A + O = A = O + A$ ，其中O是零矩阵（所有元素均为0的矩阵）。

- 零矩阵在矩阵加法中起着类似于数0在数的加法中的作用，即任何矩阵与零矩阵相加都等于该矩阵本身。

4. **负矩阵与减法**： $A - B = A + (-B)$ ，其中 $-B$ 是 B 的负矩阵（即 B 中每个元素取反得到的矩阵）。

- 矩阵的减法可以通过加法与负矩阵来实现。

矩阵的数乘**定义**：

设 k 是一个数， A 是一个矩阵，则数 k 与矩阵 A 的数乘是一个新的矩阵，记为 kA 或 Ak ，其元素由 A 中每个元素乘以 k 得到。

运算法则与数学公式：

1. **结合律**： $(kl)A = k(lA)$ ，其中 k 和 l 是数， A 是矩阵。
 - 这意味着数的乘法可以分配到矩阵的乘法中去。
2. **分配律**： $k(A + B) = kA + kB$ ，其中 k 是数， A 和 B 是矩阵。
 - 类似于数的乘法分配律，数的乘法也可以分配到矩阵的加法中去。
3. **数乘恒等元**： $1A = A$ ，其中 1 是单位数（在实数中即为 1 ）。
 - 单位数乘以任何矩阵都等于该矩阵本身。
4. **零数乘**： $0A = O$ ，其中 0 是零数（在实数中即为 0 ）， O 是零矩阵。
 - 零数乘以任何矩阵都等于零矩阵。

矩阵的乘法，听起来像是数学里的超级英雄大战，但其实它更像是两个舞者在舞台上优雅地合作完成一套复杂的舞步。想象一下，你有两个矩阵， A 和 B ，它们不是随便站着的，而是各自有着自己的“队形”（行和列）。矩阵的乘法，就是让 A 的“队员”（行）和 B 的“队员”（列）——配对，然后他们手拉手跳起舞来，最后形成一个新的队形——也就是结果矩阵 C 。

不过，这里有个小规矩： A 的队员数量（行数）得和 B 的列数相匹配，这样他们才能手牵手。如果 A 有 m 行 n 列， B 有 n 行 p 列，那么他们跳完舞后形成的新队形 C 就会有 m 行 p 列。

怎么跳这个舞呢？其实每个“舞步”都是一个数乘法和加法的过程。 A 的每一行都会和 B 的每一列进行“对话”，具体来说，就是 A 的某一行的每个元素都会和 B 的某一列的对应元素相乘，然后这些乘积加起来，就得到了 C 中对应位置的一个元素。

在NumPy里实现这个“舞蹈”超级简单，你只需要用一个星号 $*$ （但注意，这里的 $*$ 在NumPy里对于矩阵来说其实是元素对元素的乘法，而不是矩阵乘法。矩阵乘法在NumPy里用 $@$ 符号或者`numpy.dot()`函数）。不过，为了避免混淆，我们通常会使用 $@$ 符号或者`numpy.matmul()`函数来进行矩阵乘法。

举个例子：

```
import numpy as np

# 创建两个矩阵A和B
# 假设A是2x3的矩阵，B是3x2的矩阵，这样它们就可以相乘了
A = np.array([[1, 2, 3],
               [4, 5, 6]])
B = np.array([[7, 8],
               [9, 10],
               [11, 12]])

# 使用@符号进行矩阵乘法
C = A @ B
# 或者使用numpy.matmul()函数，效果一样
# C = np.matmul(A, B)

print(C)
# 输出将会是：
# [[ 58  64]
#  [139 154]]
# 这就是A和B“跳舞”后的新队形C
```

看，就像变魔术一样，两个矩阵A和B通过矩阵乘法“跳”出了一个全新的矩阵C！在NumPy的帮助下，这个过程变得既简单又有趣，就像是在数学的乐园里玩了一场别开生面的“舞蹈游戏”。

刚刚的例子等于是以讲故事的方式告诉大家怎么进行矩阵乘法。矩阵的乘法是线性代数中的一个基本且重要的运算，它遵循特定的规则来生成一个新的矩阵。以下是矩阵乘法的定义：

定义：

设 A 是一个 $m \times n$ 矩阵，其元素为 a_{ij} （其中 $i = 1, 2, \dots, m$ 且 $j = 1, 2, \dots, n$ ）， B 是一个 $n \times p$ 矩阵，其元素为 b_{jk} （其中 $j = 1, 2, \dots, n$ 且 $k = 1, 2, \dots, p$ ）。则矩阵 A 与矩阵 B 的乘积 $C = AB$ 是一个 $m \times p$ 矩阵，其元素 c_{ik} 定义为：

$$c_{ik} = \sum_{j=1}^n a_{ij} b_{jk} \quad (i = 1, 2, \dots, m; k = 1, 2, \dots, p)$$

这里， $\sum_{j=1}^n$ 表示对 j 从 1 到 n 的所有值求和。换句话说， c_{ik} 是通过取 A 的第 i 行与 B 的第 k 列的对应元素相乘后的和得到的。也就是：

\$\$

$$C = AB = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$
$$\begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{bmatrix}$$

$$\begin{bmatrix} \sum_{j=1}^n a_{1j} b_{j1} & \sum_{j=1}^n a_{1j} b_{j2} & \cdots & \sum_{j=1}^n a_{1j} b_{jp} \\ \sum_{j=1}^n a_{2j} b_{j1} & \sum_{j=1}^n a_{2j} b_{j2} & \cdots & \sum_{j=1}^n a_{2j} b_{jp} \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{j=1}^n a_{mj} b_{j1} & \sum_{j=1}^n a_{mj} b_{j2} & \cdots & \sum_{j=1}^n a_{mj} b_{jp} \end{bmatrix}$$

\$\$

注意：在公式中，矩阵 A 和 B 的乘积 C 的每个元素 c_{ik} 是通过内层求和（对 j 的求和）得到的，这反映了矩阵乘法的定义。此外，矩阵 C 的维度 $(m \times p)$ 由 A 的行数 m 和 B 的列数 p 决定，而 A 的列数 n 和 B 的行数 n 必须相等，以确保乘法运算的合法性。

2. 矩阵的逻辑运算——查找

当然了，在矩阵这种多维形式的数组下，除了执行算术运算，也可以进行增删改查这些操作。多维数组的筛选与一维数组类似，但你可以使用更复杂的条件，并且可以利用多维索引。在上面的例子中，我们

已经看到了如何使用 `[:, 0]` 和 `[:, 1]` 来索引二维数组的行和列。

```
# 假设我们有一个表示学生分数的二维数组
scores = np.array([[85, 92], [78, 85], [90, 95], [55, 60]])

# 筛选数学成绩（第二列）大于80的所有行
high_math_scores = scores[scores[:, 1] > 80]
print("Students with high math scores:", high_math_scores)
```

`np.where()` 函数是NumPy中非常实用的一个函数，它可以根据条件表达式返回满足条件的元素的索引或者根据条件从数组中选择元素。

`np.where(condition, [x, y])` 函数有三个参数，但通常我们只使用前两个参数：

- `condition`：条件表达式，返回一个布尔数组。
- `[x, y]`（可选）：如果提供了这两个参数，`np.where()` 将返回两个数组，第一个数组包含 `condition` 为True的元素的索引，第二个数组包含 `condition` 为False的元素的索引。如果只提供了 `condition` 参数，则 `np.where()` 仅返回满足条件的元素的索引。

如果不提供 `[x, y]` 参数，`np.where()` 返回的是一个元组，其中包含满足条件的元素的索引（以数组的形式）。

假设我们有一个数组，并且想要找到所有大于某个值的元素的索引，或者根据条件选择元素。

```

import numpy as np

# 创建一个数组
arr = np.array([1, 2, 3, 4, 5, 6])

# 使用 np.where() 找到所有大于3的元素的索引
indices = np.where(arr > 3)
print("Indices of elements greater than 3:", indices)

# 如果你想要基于条件获取实际的元素值
filtered_values = arr[arr > 3]
print("Values greater than 3:", filtered_values)

# 如果你想要同时获取True和False条件下的索引（尽管这在这个场景下可能不常见）
true_indices, false_indices = np.where(arr > 3, [True_indices, False_indices])
# 注意：上面的代码实际上不会按预期工作，因为np.where()不是设计来这样使用的。
# 正确的做法是仅使用condition参数，或者如果你有x和y数组，则使用它们来根据条件替换值。

# 一个更实际的例子，使用x和y来替换值
arr_replaced = np.where(arr > 3, 'High', 'Low')
print("Replaced array with 'High' and 'Low':", arr_replaced)
# 注意：这里arr_replaced的类型将是'object'，因为NumPy无法推断出所有元素都是字符串

# 对于多维数组
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
# 找到所有第二列大于5的元素的行索引和列索引
rows, cols = np.where(arr_2d[:, 1] > 5)
print("Rows and cols where second column > 5:", rows, cols)

```

在上面的示例中，我们展示了如何使用 `np.where()` 来找到满足条件的元素的索引，以及如何使用它来根据条件替换数组中的值。对于多维数组，我们可以使用多维索引来指定条件。

注意，在多维数组的示例中，`np.where()` 返回了两个数组：第一个数组包含满足条件的元素的行索引，第二个数组包含列索引。这是因为 `np.where()` 总是返回一个元组，其元素数量与条件数组中的维度数量相匹配。

3. 矩阵上的统计

NumPy还提供了丰富的函数来计算数组的统计指标，包括平均值（`mean()`）、和（`sum()`）、累积和（`cumsum()`）、连乘积（`prod()`）、累计积（`cumprod()`）、中位数（`median()`）、标准差（`std()`）和方差（`var()`）等。这些函数不仅提高了数据处理的效率，还使得数据分析变得更加简单。

和直观。下面通过一个具体的案例来展示NumPy中 `mean()` , `sum()` , `cumsum()` , `prod()` , `cumprod()` , `median()` , `std()` , 和 `var()` 这些统计指标计算方法的用法与输出结果。

假设我们有一个二维数组 `arr` , 表示某个班级三位同学的三门课成绩:

```
import numpy as np

# 创建一个二维数组, 表示三位同学的三门课成绩
arr = np.array([[85, 92, 88],
                [78, 90, 85],
                [90, 87, 92]])
```

1. `mean()`

计算所有成绩的平均值:

```
mean_all = np.mean(arr)
print("所有成绩的平均值:", mean_all) # 输出: 所有成绩的平均值: 87.5
```

计算每门课的平均成绩:

```
mean_each_subject = np.mean(arr, axis=0)
print("每门课的平均成绩:", mean_each_subject) # 输出: 每门课的平均成绩: [84.33333333 89.66666667]
```

2. `sum()`

计算所有成绩的总和:

```
sum_all = np.sum(arr)
print("所有成绩的总和:", sum_all) # 输出: 所有成绩的总和: 262
```

计算每门课的总成绩:

```
sum_each_subject = np.sum(arr, axis=0)
print("每门课的总成绩:", sum_each_subject) # 输出: 每门课的总成绩: [262 269 265] 注意这里应该是[262 269 265]
```

3. `cumsum()`

计算成绩的累积和 (按行):

```
cumsum_rows = np.cumsum(arr, axis=1)
print("按行的累积和:", cumsum_rows)
# 输出:
# [[ 85 177 265]
#   [ 78 168 253]
#   [ 90 177 269]]
```

4. prod()

计算所有成绩的连乘积:

```
prod_all = np.prod(arr)
print("所有成绩的连乘积:", prod_all) # 输出: 所有成绩的连乘积: 5217120
```

计算每位同学的总成绩连乘积 (实际上不常用, 但为展示用法):

```
prod_each_student = np.prod(arr, axis=1)
print("每位同学的总成绩连乘积:", prod_each_student) # 输出: 每位同学的总成绩连乘积: [73024 58140 685056]
```

5. cumprod()

计算成绩的累积积 (按行):

```
cumprod_rows = np.cumprod(arr, axis=1)
print("按行的累积积:", cumprod_rows)
# 输出:
# [[ 85 7225 633800]
#   [ 78 6240 530400]
#   [ 90 7830 685056]]
```

6. median()

计算所有成绩的中位数:

```
median_all = np.median(arr)
print("所有成绩的中位数:", median_all) # 输出: 所有成绩的中位数: 88.0
```

计算每门课的中位数成绩 (注意, 这里为了演示, 我们将其转换为一维数组):


```
median_each_subject = np.median(arr.ravel()) # ravel()将数组展平
print("每门课的中位数成绩:", median_each_subject) # 注意: 这实际上是所有成绩的中位数, 因为展平了
# 若要计算每门课的中位数, 应分别处理每列
median_each_subject_correct = np.median(arr, axis=0)
print("正确的每门课的中位数成绩:", median_each_subject_correct) # 输出:
```

4. 矩阵的结果保存

矩阵可能规模很大, 那如何把来之不易计算得到的结果保存起来呢? 我们看一个例子:

首先, 我们进行两个数组的加法运算, 然后将结果保存到一个文本文件中。

```
import numpy as np

# 创建两个数组
array1 = np.array([1, 2, 3, 4, 5])
array2 = np.array([5, 4, 3, 2, 1])

# 进行数组加法运算
result_array = array1 + array2

# 打印结果, 查看加法运算后的数组
print("加法运算结果: ", result_array)

# 使用np.savetxt保存结果到文本文件
# 假设我们要将结果保存到名为'result.txt'的文件中
np.savetxt('result.txt', result_array, fmt='%d', delimiter=',', newline='\n')

# 注意: fmt参数指定了每个元素的格式, 这里是整数'%d'
# delimiter参数指定了元素之间的分隔符, 这里是逗号','
# newline参数指定了每行结束后的换行符, 这里是换行符'\n'
# 这些参数可以根据需要进行调整
```

在上述例子中, 我们首先创建了两个一维数组 `array1` 和 `array2`, 并进行了加法运算, 得到了结果数组 `result_array`。然后, 我们使用 `np.savetxt` 函数将 `result_array` 保存到名为 `result.txt` 的文本文件中。在 `np.savetxt` 函数中, 我们通过 `fmt` 参数指定了每个元素的格式 (这里是整数格式 `'%d'`), 通过 `delimiter` 参数指定了元素之间的分隔符 (这里是逗号,), 以及通过 `newline` 参数指定了每行结束后的换行符 (这里是换行符 `\n`)。

`np.savetxt` 是一个非常灵活的函数, 支持多种参数来定制输出文件的格式。除了上面提到的 `fmt`、`delimiter` 和 `newline` 参数外, 还有一些其他常用的参数:

- header：用于在文件顶部添加注释或标题行。
- footer：用于在文件底部添加文本。
- comments：指定注释字符，默认为 '#'。如果 header 或 footer 参数中包含该字符，它们将被视为注释。

例如，我们可以向文件添加标题和注释：

```
np.savetxt('result_with_header.txt', result_array, fmt='%d', delimiter=',',  
           header='这是加法运算的结果', comments='# ')
```

这样，在 result_with_header.txt 文件的顶部将会有有一个注释行 # 这是加法运算的结果。

如果想要打开，只需要使用loadtxt进行开启即可：

```
# 使用loadtxt加载之前保存的数组  
# 根据保存时指定的格式和分隔符来加载  
loaded_arr = np.loadtxt('array_data.txt', delimiter=',', dtype=float)  
  
# 如果你保存时没有指定fmt或者delimiter，这里也需要相应地调整  
# 比如，如果默认分隔符是空格，且没有指定fmt  
# loaded_arr = np.loadtxt('array_data_no_fmt.txt', dtype=float)  
  
# 打印加载的数组来验证  
print(loaded_arr)
```

通过调整这些参数，我们可以轻松地根据需保存数组数据到文本文件中，便于后续的分析或共享。